Double Agent Dilemma

- **Time limit:** 12 minutes.
- **Storage:** 5 GB
- **Environment:** one GPU (≈16 GB VRAM), no internet
- **Solution size:** `solution.ipynb` ≤ 1 MB
- **Baseline score:** 0

At the national AI center in Astana, two computer models — Model R (a ResNet-18) and Model V (a ViT-Tiny) —are analyzing photos. Right now, both models are doing a perfect job, scoring 100% accuracy and agreeing on every single image. To test how different their smart "brains" really are, the chief scientist gives you a challenge: make tiny, almost invisible pixel changes to each photo so that Model R and Model V completely disagree.

1. Task

Two pretrained image classifiers look at the same image. On the images provided in this task, both classifiers perform with 100% accuracy.

- **Model R**: `torchvision.models.resnet18` (a CNN, ResNet18).
- **Model V**: `timm`'s `vit_tiny_patch16_224` (a Transformer, ViT-Tiny).

Your task is to create a small change ("perturbation") for each image so that the two models disagree. For every image, you must create **two different** perturbations:

- **Type A**: after adding it, Model R still classifies the image correctly, but Model V classifies it incorrectly.
- **Type B**: after adding it, Model V still classifies the image correctly, but Model R classifies it incorrectly.

Each perturbation must be *small* enough that it is hard to notice. Smaller perturbations score higher (see Section 5). The perturbation is applied to the original image directly on the pixel level.

2. Public data

A set of images is provided with the task, organized into two splits — `train` (100 images) and `test_public` (100 images) — each with images of varying resolution. All images are from ImageNet-1K's 1000 classes and both Model R and Model V achieve 100% accuracy on both splits.

The following files are provided:

```
train/images/*.png          # 100 images in PNG format
train/labels.json           # maps each image index to its correct class
test_public/images/*.png    # 100 images in PNG format
test_public/labels.json     # maps each image index to its correct class
```

During grading time, your `dataset/test_public/` folder is transparently replaced by two hidden sets of images (`test_leaderboard_a` and `test_leaderboard_b`) for official scoring. Each of them contains **100 images** in PNG format and a label file.

Note: For this task, the labels in test datasets are accessible.

3. Output format

For each image, you must produce two files:

```
{index}_a.pt   # Type A perturbation
{index}_b.pt   # Type B perturbation
```

- `{index}` (`0`, `1`, `2`, ...), matches the image's name in the datasets.
- Each file is a single tensor saved with `torch.save`. Its shape must be `3 x H x W`, where `H` and `W` match the **original** resolution of that image (not `224 x 224`).
- The code should produce only one ZIP file, `submission.zip`. Place all `.pt` files at the top level of the ZIP archive, with no enclosing folder or subdirectories.

```
submission.zip
├── 0_a.pt
├── 0_b.pt
└── 1_a.pt
└── ...
```

The notebook will alert you if there is any issues with the output format.

4. Constraints

- **Models:** You must use `torchvision.models.resnet18(pretrained=True)` and `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. No other pretrained models are allowed.
- **Transform pipeline (enforced at evaluation):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])`. See Section 3 of `baseline.ipynb` for details.
- **Perturbation resolution:** Must match the **original** raw image resolution (not 224×224). The tensor is added to the raw image *before* the transform pipeline.
- **Output format:** `.pt` files only — no PNG/JPG . Tensors are added to the image (normalized to `[0,1]`) and then values are clipped to `[0, 1]` before preprocessing.
- **File naming:** Flat-listed, strict `{index}_a.pt` / `{index}_b.pt` format. No subdirectories inside the zip.
- **Libraries:** `torch`, `torchvision`, `timm`.

5. Scoring

The final score is computed as follows. Let `M` be the number of images in the split, $Score_A$ the number of successful Type A perturbations, and $Score_B$ the number of successful Type B perturbations:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF is a function designed to penalise perturbations with a high norm and to be very sensitive near the ceiling of performance. It it bound in the range 0.5 to 1. The full implementation can be seen in Section 8 of `solution.ipynb`.

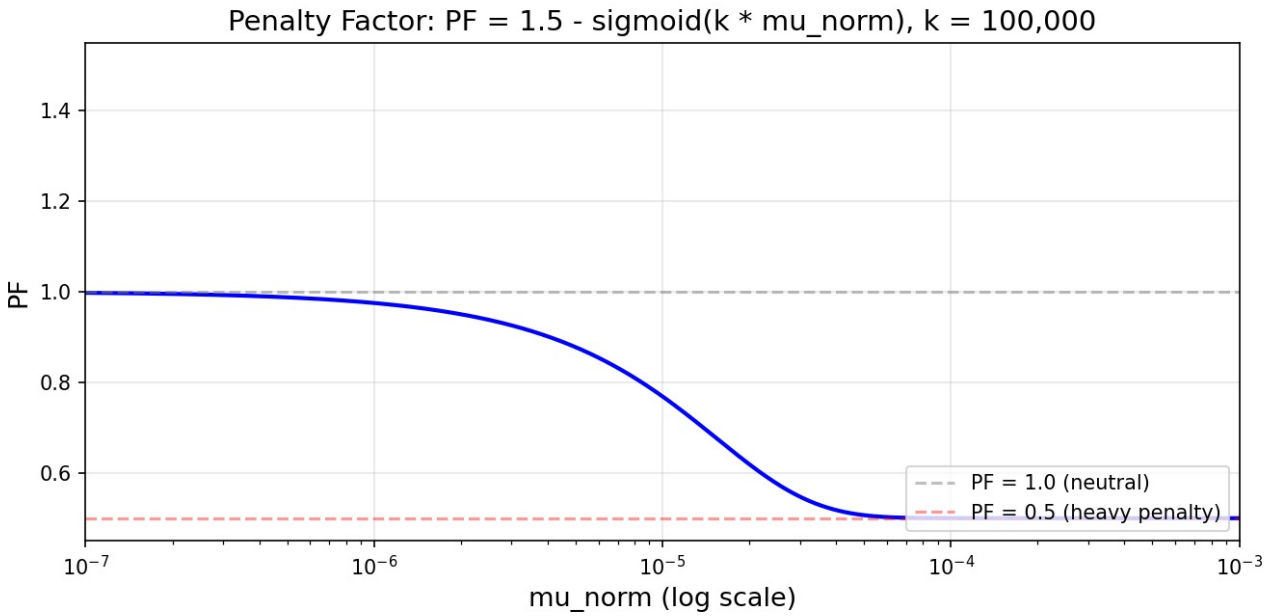

Figure: The curve of the penalty function.

6. Check the Submission

There are checks in the notebook that alert you if there are formatting issues, at Section 7 in the `solution.ipynb` notebook.

7. Local testing

`solution.ipynb` contains a complete, working example. It loads the public data, both models, and the official scorer, and writes a submission ZIP file. Read it before you start.

8. How to submit

- Save your changes to `solution.ipynb`.
- Open the Git tab in the left sidebar of JupyterLab.
- **Stage** `solution.ipynb` (the + icon next to it).
- Enter a commit message and click **Commit**.
- Click the cloud-with-up-arrow to push.
- Return to this Contest page and click **Submit**.

Submit exactly one file, named `solution.ipynb`.